

Extended Artifact Description

Artifact Description (AD)

I. OVERVIEW OF CONTRIBUTIONS AND ARTIFACTS

A. Paper's Main Contributions

In the paper titled "Fantastic Hardware Counters and How to Use Them for Noise-Resilient Performance Modeling," we present a toolchain for identifying hardware performance counters that are resilient against noise and suitable for modeling. Followed by a demonstration of this toolchain. In particular, our main contributions were:

- C_1 Provide an approach integrated into NORC that automatically identifies noise-resilient hardware counters suitable for performance modeling, reducing model error by up to 75% over time-based models.
- C_2 Extending Extra-P with the ability to generate performance models based on noise-resilient hardware counters.
- C_3 Demonstrate our approach based on experiments on both large and small systems, highlighting the ease of use.

B. Computational Artifacts

The artifacts of this paper comprise the results produced by the NORC and Extra-P tools for evaluating hardware performance counters and their modeling capabilities on two different systems. All artifacts are gathered and described in a single Git repository¹. Below, we list the artifacts, which are available via Zenodo², when not otherwise stated, and describe their relation to the contributions.

- A_1 Artifact A_1 is the extended NORC³ version, including the enhanced ranking with cutoff detection and the modeling-quality module. It is the contribution C_1 and used for C_3 . It is used to produce the Figs. 1–4 and influences Figs. 5–8. Furthermore, it produces the results shown in Tables II and III.
- A_2 Artifact A_2 is the extended Extra-P version with hardware counter modeling⁴, including the enhanced ranking with cutoff detection and the modeling-quality module. It is the contribution C_2 and used for C_3 . It produces the results summarized in Figs. 5–8.
- A_3 Artifact A_3 is the hardware-counter data obtained on the Lichtenberg II system for the demonstration C_3 collected and analyzed with C_1 . It shows how NORC can be used to identify noise-resilient hardware counters and automatically detect the cutoff. It is used to create Fig. 1a, and it influences Figs. 2, 4a, 5, 6, 7, and 8 as well as Tables II and III.
- A_4 Artifact A_4 is the hardware-counter data obtained on the GH system in the previous paper and is available

at Zenodo⁵. It is used similarly to Artifact A_3 for the demonstration C_3 , analyzed with C_1 . It is used to create Fig. 1b, and it influences Figs. 3, 4b, 5, 6, 7, and 8 as well as Tables II and III.

- A_5 Artifact A_5 is the hardware-counter data obtained on the Lichtenberg II system via MQM for the demonstration C_3 collected and analyzed with C_1 and C_2 . It shows how NORC can be used to identify noise-resilient hardware counters and automatically detect the cutoff. We also use it to evaluate Extra-P's modeling. It is used to create Figs. 2, 4a, 5 as well as Tables II and III.
- A_6 Artifact A_6 is the hardware-counter data obtained on the Grace Hopper system via MQM. It is used similarly to Artifact A_5 for the demonstration C_3 , analyzed with C_1 and C_2 . It is used to create Figs. 3, 4b, 5 as well as Tables II and III.
- A_7 Artifact A_7 is the measured data obtained on the Lichtenberg II system for the Kripke case study, collected for the demonstration C_3 and modeled with C_2 . It shows how Extra-P can be used to create noise-resilient hardware counters. It is used to create Figs. 6, 7.
- A_8 Artifact A_8 is the measured data obtained on the Lichtenberg II system for the RELeARN case study, collected for the demonstration C_3 and modeled with C_2 . It shows how Extra-P can be used to create noise-resilient hardware counters. It is used to create Fig. 8.
- A_9 Artifact A_9 is the measured data obtained on the Grace Hopper system for the Kripke case study, collected for the demonstration C_3 and modeled with C_2 . It shows how Extra-P can be used to create noise-resilient hardware counters. It is used to create Fig. 6.
- A_{10} Artifact A_{10} is the measured data obtained on the Grace Hopper system for the RELeARN case study, collected for the demonstration C_3 and modeled with C_2 . It shows how Extra-P can be used to create noise-resilient hardware counters. It is used to create Fig. 8.

¹<https://github.com/tuda-hpclub/NORC/tree/main/artifacts/protocols26>

²<https://doi.org/10.5281/zenodo.21886223>

³<https://github.com/tuda-hpclub/NORC>

⁴<https://github.com/extra-p/extrap/tree/feature/hwc-modeling>

⁵<https://doi.org/10.5281/zenodo.16786108>

The following table summarizes the relationship between artifacts, contributions, and related paper elements.

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_3	Figs. 1–8; Tables II–III
A_2	C_1, C_2, C_3	Figs. 5–8
A_3	C_1, C_3	Fig. 1a, 2, 4a, 5–8; Tables II–III
A_4	C_1, C_3	Fig. 1b, 3, 4b, 5, 6, 8; Tables II–III
A_5	C_1, C_2, C_3	Figs. 2, 4a, 5; Tables II–III
A_6	C_1, C_2, C_3	Figs. 3, 4b, 5; Tables II–III
A_7	C_2, C_3	Figs. 6, 7
A_8	C_2, C_3	Fig. 8
A_9	C_2, C_3	Fig. 6
A_{10}	C_2, C_3	Fig. 8

II. ARTIFACT IDENTIFICATION

A. Computational Artifact A_1

Relation To Contributions

NORC (A_1) is the tool underlying contribution C_1 ; it also produces the counter data A_3 – A_6 used for the demonstration in C_3 .

Expected Results

Running NORC on Lichtenberg II and on the Grace Hopper node should reproduce the noise-resilient hardware-counter rankings (with automatic cutoff detection) shown in Fig. 1 and via the modeling-quality module together with Extra-P (A_2), the counter recommendations in Figs. 2–4 and Tables II–III used to produce A_5/A_6 and summarized in Figs. 5–8. This substantiates C_1 (automatic identification of noise-resilient counters, reducing model error by up to 75% over time-based models) and, together with Extra-P (A_2), C_3 (demonstration on both a large and a small system).

Expected Reproduction Time (in Minutes)

Installing NORC takes around 50 min. Reproduction time otherwise depends on the target artifact; see A_3 – A_6 below for the acquisition and analysis times on each system.

Artifact Setup (incl. Inputs)

Hardware: Lichtenberg II MPI partition node ($2 \times$ Intel Xeon Platinum 8470Q, 52 cores each, 512 GB DDR5-4800, InfiniBand HDR100) and a Grace Hopper Superchip node (72 Neoverse V2 Armv9 cores, 480 GB LPDDR5X, NVIDIA GH200 GPU with 96 GB HBM3). Both systems must expose hardware performance counters to the user.

Software:

Score-P	8.0 or higher, with PAPI support, https://score-p.org
PAPI	https://icl.utk.edu/papi/
SIONlib	https://www.fz-juelich.de/jsc/sionlib
PyYaml	
numpy	
NOIGENA	https://codebase.helmholtz.cloud/g.corbin/noigena-tool
LAMMPS	https://www.lammps.org
LULESH	https://github.com/LLNL/LULESH
miniFE	https://github.com/Mantevo/miniFE

The analysis component can be run separately from the acquisition. It is a Python package, installable via `pip install`, that depends on the following packages, all available via PyPI:

- numpy
- matplotlib
- pyside6
- termcolor
- pycubexr
- tqdm
- extrap

The modeling-quality module is implicitly installed once both the acquisition and analysis components are installed.

Datasets / Inputs: None; NORC produces the datasets A_3 – A_6 described below rather than consuming pre-existing ones.

Installation and Deployment: NORC is set up on Lichtenberg II (for A_3, A_5) as follows:

- Step 1 Install and configure NORC by copying the NORC folder to your system and running the `install` script in the acquisition folder. The interactive installation and configuration assistant will guide you through the process and install all dependencies.
- Step 2 When asked if the configuration is updated for this system, choose no (N).
- Step 3 Enable the usage of the module system by choosing *Yes*.
- Step 4 Set the modules to use: `gcc/13.4.0 openmpi/4.1.8 SIONlib/1.7.7 cmake/4.2.3 scorep/9.4 papi/7.2.0`.
- Step 5 Set the number of parallel build jobs to 96.
- Step 6 When SLURM is detected, disable the local execution.
- Step 7 Select the system configuration *lichtenberg*.
- Step 8 Set the number of threads per rank to 52.
- Step 9 Choose *not* to edit the default configuration.
- Step 10 When asked if hardware counters are already configured, choose no (N).
- Step 11 Leave all of them selected, and accept.

On the Grace Hopper system (for A_4, A_6), we used the automatic installation of dependencies via Spack. GCC (version 11.4.0), make, and Python (version 3.11.3) are also needed to compile and run the experiments. The installation works as follows:

- Step 1 Install and configure NORC by copying the NORC folder to your system and running the `install` script

in the acquisition folder. The interactive installation and configuration assistant will guide you through the process and install all dependencies.

- Step 2 When asked if the configuration is updated for this system, choose no (N).
- Step 3 Enable the usage of Spack, by choosing *Yes*.
- Step 4 Set the Spack suffix to `%gcc@11.4.0`.
- Step 5 Set the number of parallel build jobs to 72.
- Step 6 Confirm that no SLURM is detected.
- Step 7 Select the *local* system configuration.
- Step 8 Set the number of threads per rank to 9.
- Step 9 Edit the default configuration to match the change in available resources (see `experiments.cfg`).
- Step 10 When asked if hardware counters are already configured, choose no (N).
- Step 11 Choose all hardware counters.

Artifact Execution

Up to four tasks need to be executed for the ranking-based artifacts (A_3 , A_4): T_1 runs the experiments, T_2 copies the results, $T_{3a,3b}$ perform the analysis, and $T_{4a,4b}$ export tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow (T_{3a} \vee T_{3b}) \rightarrow (T_{4a} \vee T_{4b})$.

- T_1 After installing and configuring NORC, run the experiments with the `run` command to collect the raw counter data.
- T_2 Copy the experiment folder to the machine you want to run the analysis on.
- T_{3a} Open the experiment folder in the NORC analysis GUI. If prompted to start the pre-processing, select yes and wait for it to finish.
- T_{3b} Alternatively, run the analysis manually using `norc_analyze`.
- T_{4a} Export the ranking, including cutoff, of all measured counters using `norc_rank`.
- T_{4b} View the ranking plot in the GUI.

Up to three tasks need to be executed for the modeling-quality artifacts (A_5 , A_6): T_1 finds the relevant counters and runs the experiments, T_2 performs the analysis, and T_3 exports tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow T_3$.

- T_1 After installing, configuring, and running the NORC acquisition, run the MQM analysis with the `run_mq_analysis` command to find the noise-resilient counters using the NORC ranking, and then start, for those counters, a measurement job for collecting performance model data.
- T_2 When the job is done, the automated modeling-quality analysis runs, using Extra-P to find the best-fitting performance models across ranks and output the set of recommended counters.
- T_3 Use `compare_to_ground_truth.py` with the respective plotting parameter to generate the correlation heat map with violin plots as well as the clustering plots.

Artifact Analysis (incl. Outputs)

For A_3 , A_4 , the outputs are the exported ranking with cutoff of all measured counters (Figs. 1–4, Tables II and III) and the corresponding ranking plot. For A_5 , A_6 , the outputs are the set of counters recommended by the modeling-quality analysis together with the correlation heat map, violin plots, and clustering plots (Figs. 5–8, Tables II–III).

B. Computational Artifact A_2

Relation To Contributions

Extra-P with hardware-counter modeling (A_2) is contribution C_2 and is used, together with NORC (A_1), for the demonstration C_3 .

Expected Results

Extra-P should reproduce the performance models built from noise-resilient hardware counters summarized in Figs. 5–8, confirming that the extended modeler finds well-fitting models for the counters that NORC recommends.

Expected Reproduction Time (in Minutes)

Installation takes only a few minutes; reproduction time is otherwise dominated by the NORC acquisition and analysis steps of A_5 , A_6 described above (around 540 min and 320 min, respectively, plus around 10 min analysis).

Artifact Setup (incl. Inputs)

Hardware: No specific hardware is required beyond a machine capable of running Python 3.10+; the counter data it models was collected on the Lichtenberg II and Grace Hopper systems described for A_1 .

Software: Extra-P⁶ requires Python 3.10 or later. The following modules are automatically retrieved and installed via pip:

- numpy
- matplotlib
- tqdm
- pycubexr
- marshmallow
- packaging
- kaitaistruct
- protobuf
- itanium_demangler
- sympy
- typing-extensions
- msgpack
- scipy
- pyside6
- pyvista
- pyvistaqt
- pyobjc-framework-Cocoa when used on macOS

Datasets / Inputs: Extra-P models the noise-resilient hardware-counter data produced by Score-P instrumented codes (A_5 – A_{10}); no separate download is required.

⁶<https://github.com/extra-p/extrap/tree/feature/hwc-modeling>

Installation and Deployment: Install the extended Extra-P version via `pip install`, which resolves and installs all dependencies listed above automatically. No separate compilation step is required.

Artifact Execution

Extra-P is invoked as part of T_2 of the A_5 , A_6 workflow above: after the NORC-recommended measurement job completes, the automated modeling-quality analysis calls Extra-P to fit performance models across ranks for the candidate counters. For A_7 to A_{10} Extra-P generates the performance models based on the profiles of the evaluated applications.

Artifact Analysis (incl. Outputs)

The output is the set of performance models based on runtime and noise-resilient counters, summarized in Figs. 5–8 and Tables II and III.

C. Computational Artifact A_3

Relation To Contributions

Artifact A_3 is the hardware counter data obtained on the Lichtenberg II system for the demonstration C_3 , collected and analyzed with C_1 using NORC (A_1).

Expected Results

Running NORC on Lichtenberg II should reproduce the noise-resilient hardware-counter ranking (with automatic cutoff detection) shown in Fig. 1a and Tables II–III, and influencing Figs. 2, 4a, and 5–8. This substantiates C_1 and, together with Extra-P (A_2), C_3 .

Expected Reproduction Time (in Minutes)

Data acquisition: around 300 min (add 50 min if NORC itself still needs to be installed). Analysis: around 180 min on a common laptop.

Artifact Setup (incl. Inputs)

Hardware: Lichtenberg II MPI partition node (2× Intel Xeon Platinum 8470Q, 52 cores each, 512 GB DDR5-4800, InfiniBand HDR100). The system must expose hardware performance counters to the user.

Software: NORC (A_1) and its dependencies (Score-P, PAPI, SIONlib, PyYaml, numpy, NOIGENA, LAMMPS, LULESH, miniFE, and the NORC analysis Python package); see A_1 for details.

Datasets / Inputs: A_3 is the raw and processed hardware-counter measurements collected by NORC; it is generated by running the acquisition and analysis workflow below rather than downloaded, and the resulting dataset is archived on Zenodo (see Section I).

Installation and Deployment: NORC is set up on Lichtenberg II as follows:

- Step 1 Install and configure NORC by copying the NORC folder to your system and running the `install` script in the acquisition folder. The interactive installation and configuration assistant will guide you through the process and install all dependencies.
- Step 2 When asked if the configuration is updated for this system, choose no (N).
- Step 3 Enable the usage of the module system by choosing *Yes*.
- Step 4 Set the modules to use: `gcc/13.4.0 openmpi/4.1.8 SIONlib/1.7.7 cmake/4.2.3 scorep/9.4 papi/7.2.0`.
- Step 5 Set the number of parallel build jobs to 96.
- Step 6 When SLURM is detected, disable the local execution.
- Step 7 Select the system configuration *lichtenberg*.
- Step 8 Set the number of threads per rank to 52.
- Step 9 Choose *not* to edit the default configuration.
- Step 10 When asked if hardware counters are already configured, choose no (N).
- Step 11 Leave all of them selected, and accept.

Artifact Execution

Up to four tasks need to be executed: T_1 runs the experiments, T_2 copies the results, $T_{3a,3b}$ perform the analysis, and $T_{4a,4b}$ export tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow (T_{3a} \vee T_{3b}) \rightarrow (T_{4a} \vee T_{4b})$.

- T_1 After installing and configuring NORC, run the experiments with the `run` command to collect the raw counter data.
- T_2 Copy the experiment folder to the machine you want to run the analysis on.
- T_{3a} Open the experiment folder in the NORC analysis GUI. If prompted to start the pre-processing, select yes and wait for it to finish.
- T_{3b} Alternatively, run the analysis manually using `norc_analyze`.
- T_{4a} Export the ranking, including cutoff, of all measured counters using `norc_rank`.
- T_{4b} View the ranking plot in the GUI.

Artifact Analysis (incl. Outputs)

The outputs are the exported ranking with cutoff of all measured counters (Fig. 1a, Tables II and III) and the corresponding ranking plot.

D. Computational Artifact A_4

Relation To Contributions

Artifact A_4 is the hardware counter data obtained on the Grace Hopper (GH) system in the previous paper, available at Zenodo⁷. It is used, like A_3 , for the demonstration C_3 , analyzed with C_1 .

⁷<https://doi.org/10.5281/zenodo.16786108>

Expected Results

Running NORC on the Grace Hopper node should reproduce the noise-resilient hardware-counter ranking (with automatic cutoff detection) shown in Fig. 1b and Tables II–III, and influencing Figs. 3, 4b, and 5–8. This substantiates C_1 and, together with Extra-P (A_2), C_3 .

Expected Reproduction Time (in Minutes)

Data acquisition: around 2200 min (add 50 min if NORC itself still needs to be installed). Analysis: around 180 min on a common laptop.

Artifact Setup (incl. Inputs)

Hardware: A Grace Hopper Superchip node (72 Neo-verse V2 Armv9 cores, 480 GB LPDDR5X, NVIDIA GH200 GPU with 96 GB HBM3). The system must expose hardware performance counters to the user.

Software: NORC (A_1) and its dependencies (Score-P, PAPI, SIONlib, PyYaml, numpy, NOIGENA, LAMMPS, LULESH, miniFE, and the NORC analysis Python package); see A_1 for details. GCC (version 11.4.0), make, and Python (version 3.11.3) are additionally needed to compile and run the experiments.

Datasets / Inputs: A_4 is the raw and processed hardware-counter measurements collected by NORC on the Grace Hopper system; it is generated by running the acquisition and analysis workflow below. It has been collected in a previous paper. The dataset is archived on Zenodo (see above).

Installation and Deployment: On the Grace Hopper system, we used the automatic installation of NORC’s dependencies via Spack. The installation works as follows:

- Step 1 Copy the NORC folder to your system and run the `install` script in the acquisition folder to start the interactive installation and configuration assistant.
- Step 2 When asked if the configuration is updated for this system, choose no (N).
- Step 3 Enable the usage of Spack, by choosing *Yes*.
- Step 4 Set the Spack suffix to `%gcc@11.4.0`.
- Step 5 Set the number of parallel build jobs to 72.
- Step 6 Confirm that no SLURM is detected.
- Step 7 Select the *local* system configuration.
- Step 8 Set the number of threads per rank to 9.
- Step 9 Edit the default configuration to match the change in available resources (see `experiments.cfg`).
- Step 10 When asked if hardware counters are already configured, choose no (N).
- Step 11 Choose all hardware counters.

Artifact Execution

Up to four tasks need to be executed: T_1 runs the experiments, T_2 copies the results, $T_{3a,3b}$ perform the analysis, and $T_{4a,4b}$ export tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow (T_{3a} \vee T_{3b}) \rightarrow (T_{4a} \vee T_{4b})$.

- T_1 After installing and configuring NORC, run the experiments with the `run` command to collect the raw counter data.

- T_2 Copy the experiment folder to the machine you want to run the analysis on.
- T_{3a} Open the experiment folder in the NORC analysis GUI. If prompted to start the pre-processing, select yes and wait for it to finish.
- T_{3b} Alternatively, run the analysis manually using `norc_analyze`.
- T_{4a} Export the ranking, including cutoff, of all measured counters using `norc_rank`.
- T_{4b} View the ranking plot in the GUI.

Artifact Analysis (incl. Outputs)

The outputs are the exported ranking with cutoff of all measured counters (Fig. 1b, Tables II and III) and the corresponding ranking plot.

E. Computational Artifact A_5

Relation To Contributions

Artifact A_5 is the hardware counter data obtained on the Lichtenberg II system via the modeling-quality module (MQM) for the demonstration C_3 , collected and analyzed with C_1 and C_2 . We also use it to evaluate Extra-P’s modeling.

Expected Results

Running the NORC/MQM workflow on Lichtenberg II should reproduce the modeling-quality-based recommendation of counters shown in Figs. 2, 4a, and 5, and Tables II–III, using Extra-P (A_2) to find the best-fitting performance models. This substantiates C_1 and, together with Extra-P, C_3 .

Expected Reproduction Time (in Minutes)

Data acquisition: around 540 min (add 50 min if NORC itself still needs to be installed). Analysis: around 10 min on a common laptop.

Artifact Setup (incl. Inputs)

Hardware: Lichtenberg II MPI partition node ($2 \times$ Intel Xeon Platinum 8470Q, 52 cores each, 512 GB DDR5-4800, InfiniBand HDR100). The system must expose hardware performance counters to the user.

Software: NORC (A_1) and its dependencies, including Extra-P; see A_1 and A_2 for details.

Datasets / Inputs: A_5 is the raw and processed hardware-counter measurements and fitted performance models produced by the NORC/MQM workflow; it is generated by running the workflow below.

Installation and Deployment: NORC is set up on Lichtenberg II as described for A_3 (Steps 1–11). Extra-P is additionally installed via `pip install` as described for A_2 .

Artifact Execution

Up to three tasks need to be executed: T_1 finds the relevant counters and runs the experiments, T_2 performs the analysis, and T_3 exports tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow T_3$.

- T_1 After installing, configuring, and running the NORC acquisition, run the MQM analysis with the

`run_mq_analysis` command to find the noise-resilient counters using the NORC ranking, and then start, for those counters, a measurement job for collecting performance model data.

- T_2 When the job is done, the automated modeling-quality analysis runs, using Extra-P to find the best-fitting performance models across ranks and output the set of recommended counters.
- T_3 Use `compare_to_ground_truth.py` with the respective plotting parameter to generate the correlation heat map with violin plots as well as the clustering plots.

Artifact Analysis (incl. Outputs)

The outputs are the set of counters recommended by the modeling-quality analysis together with the correlation heat map, violin plots, and clustering plots (Figs. 2, 4a, 5, Tables II–III).

F. Computational Artifact A_6

Relation To Contributions

Artifact A_6 is the hardware counter data obtained on the Grace Hopper system via the modeling-quality module (MQM). It is used, like A_5 , for the demonstration C_3 , analyzed with C_1 and C_2 .

Expected Results

Running the NORC/MQM workflow on the Grace Hopper node should reproduce the modeling-quality-based recommendation of counters shown in Figs. 3, 4b, and 5, and Tables II–III, using Extra-P (A_2) to find the best-fitting performance models. This substantiates C_1 and, together with Extra-P, C_3 .

Expected Reproduction Time (in Minutes)

Data acquisition: around 320 min (add 50 min if NORC itself still needs to be installed). Analysis: around 10 min on a common laptop.

Artifact Setup (incl. Inputs)

Hardware: A Grace Hopper Superchip node (72 Neoverse V2 Armv9 cores, 480 GB LPDDR5X, NVIDIA GH200 GPU with 96 GB HBM3). The system must expose hardware performance counters to the user.

Software: NORC (A_1) and its dependencies, including Extra-P; see A_1 and A_2 for details. GCC (version 11.4.0), make, and Python (version 3.11.3) are additionally needed to compile and run the experiments.

Datasets / Inputs: A_6 is the raw and processed hardware-counter measurements and fitted performance models produced by the NORC/MQM workflow on the Grace Hopper system; it is generated by running the workflow below.

Installation and Deployment: NORC is set up on the Grace Hopper system as described for A_4 (Steps 1–11, via Spack). Extra-P is additionally installed via `pip install` as described for A_2 .

Artifact Execution

Up to three tasks need to be executed: T_1 finds the relevant counters and runs the experiments, T_2 performs the analysis, and T_3 exports tables and figures. The workflow is $T_1 \rightarrow T_2 \rightarrow T_3$.

- T_1 After installing, configuring, and running the NORC acquisition, run the MQM analysis with the `run_mq_analysis` command to find the noise-resilient counters using the NORC ranking, and then start, for those counters, a measurement job for collecting performance model data.
- T_2 When the job is done, the automated modeling-quality analysis runs, using Extra-P to find the best-fitting performance models across ranks and output the set of recommended counters.
- T_3 Use `compare_to_ground_truth.py` with the respective plotting parameter to generate the correlation heat map with violin plots as well as the clustering plots.

Artifact Analysis (incl. Outputs)

The outputs are the set of counters recommended by the modeling-quality analysis together with the correlation heat map, violin plots, and clustering plots (Figs. 3, 4b, 5, Tables II–III).

G. Computational Artifact A_7

Relation To Contributions

Artifact A_7 is the measured data obtained on the Lichtenberg II system for the Kripke case study, collected for the demonstration C_3 and modeled with C_2 .

Expected Results

Modeling the collected Kripke measurements with the Extra-P base and HWC modelers should reproduce the mean exponent deviation and relative-error results shown in Figs. 6–7, showing that Extra-P, combined with NORC-selected counters, produces noise-resilient performance models for real applications.

Expected Reproduction Time (in Minutes)

Data acquisition: around 7800 min. The evaluation scripts (relative error and mean exponent deviation) take roughly 30 min combined across all of A_7 – A_{10} .

Artifact Setup (incl. Inputs)

Hardware: Lichtenberg II MPI partition node ($2 \times$ Intel Xeon Platinum 8470Q, 52 cores each, 512 GB DDR5-4800, InfiniBand HDR100).

Software: Kripke⁸ (benchmark code, bundling RAJA as `tpl/raja`), Score-P (for instrumentation and profiling), and Extra-P (A_2) for modeling.

Datasets / Inputs: A_7 is generated by running the measurement orchestration scripts described below with the parameter sets and job templates provided in the dataset.

⁸<https://github.com/LLNL/Kripke>

Installation and Deployment: Kripke is compiled with the Score-P compiler wrappers so that all MPI and OpenMP regions are automatically instrumented; no source-code changes are required beyond a single patch described below. To ease reproduction, we provide a setup script (`setup_kripke_spack.sh`) alongside the dataset that automates the steps below.

Step 1 Load the required modules: `module load gcc/14 openmpi cmake scorep`.

Step 2 Point the compiler variables to the Score-P instrumented wrappers: `CC=scorep-mpicc, CXX=scorep-mpic++, FC=scorep-mpif77`.

Step 3 Apply the provided patch to `tpl/raja/include/RAJA/policy/openmp/scan.hpp`. The patch wraps the call to `exclusive_inplace` inside each `#pragma omp single` region in an explicit block; without it, only the call itself—and not the subsequent for loop—is bound to the pragma, which causes miscompilation with the Score-P instrumented compiler.

Step 4 Remove any pre-existing build directory.

Step 5 Configure the build with CMake, with Score-P instrumentation disabled for the configure step itself (`SCOREP_WRAPPER=off`) to prevent the compiler feature-detection test compiles from being instrumented:

```
SCOREP_WRAPPER=off cmake -S . -B build \
  -DCMAKE_CXX_COMPILER=scorep-mpic++ \
  -DCMAKE_C_COMPILER=scorep-mpicc \
  -DENABLE_OPENMP=1 -DENABLE_MPI=1 \
  -DRAJA_ENABLE_EXERCISES=Off
```

Step 6 Build inside `build/`, disabling Score-P's automatic OpenMP instrumentation for the compile step:

```
SCOREP_WRAPPER_INSTRUMENTER_FLAGS= \
  "--thread=none" make -j48
```

Artifact Execution

Data collection for A_7 is split into generating the measurement orchestration script and then running it. The workflow is T_1 (build & configure) $\rightarrow T_2$ (generate script) $\rightarrow T_3$ (run measurement sweep) $\rightarrow T_4$ (evaluate).

T_1 Build Kripke with the Score-P compiler wrappers as described above.

T_2 Generate the measurement orchestration script (`norc_generate`) from the `batch_template.sh` job template, using the counters from the MQM `selected_counters.json` produced for A_5 (`--counter-set both`, i.e., the union of the rule-based and clustered counter sets). Sweep the zones-per-rank parameter Z_p over `--var edge=8,16,24,32,40`, G over `--var grp=16,32,48,64,80` and the number of MPI ranks p over `--var ntasks=2,4,8,16,32`, with a single repetition per combination of counters (`--iterations 1`).

T_3 Run the generated script to execute the full parameter sweep, including the held-out reference point,

collecting the raw counter data in the results directory. Additionally, measure the held-out reference point `--var edge=48, --var grp=96, --var ntasks=64`, used to evaluate the mean relative error against ground truth.

T_4 Model the results with Extra-P; specifically, run the Kripke evaluation script (`kripke_evaluation.py`) to obtain the mean exponent deviation of the models, and the `relative_error_evaluation.py` script to determine the relative errors. Both scripts run the Extra-P base and HWC modelers on the dataset.

Artifact Analysis (incl. Outputs)

The evaluation scripts produce CSV files containing the mean-exponent-deviation and relative-error data shown in Figs. 6–7.

H. Computational Artifact A_8

Relation To Contributions

Artifact A_8 is the measured data obtained on the Lichtenberg II system for the RELeARN case study, collected for the demonstration C_3 and modeled with C_2 .

Expected Results

Modeling the collected RELeARN measurements with the Extra-P base and HWC modelers should reproduce the mean exponent deviation and relative-error results shown in Fig. 8, showing that Extra-P, combined with NORC-selected counters, produces noise-resilient performance models for real applications.

Expected Reproduction Time (in Minutes)

Data acquisition: around 300 min. The evaluation scripts (relative error and mean exponent deviation) take roughly 30 min combined across all of A_7 – A_{10} .

Artifact Setup (incl. Inputs)

Hardware: Lichtenberg II MPI partition node (2× Intel Xeon Platinum 8470Q, 52 cores each, 512 GB DDR5-4800, InfiniBand HDR100).

Software: RELeARN⁹ (benchmark code), Score-P (for instrumentation and profiling), and Extra-P (A_2) for modeling.

Datasets / Inputs: A_8 is generated by running the measurement orchestration scripts described below with the parameter sets and job templates provided in the dataset.

Installation and Deployment: RELeARN is compiled with the Score-P compiler wrappers so that all MPI and OpenMP regions are automatically instrumented; no source-code changes are required. To ease reproduction, we provide a setup script (`setup_relearn_spack.sh`) alongside the dataset that automates the steps below.

Step 1 Purge any loaded modules and load the required ones: `ml purge, then ml gcc/14.3.0 openmpi/4.1.8 scorep/9.4 boost/1.89.0`.

⁹<https://github.com/tuda-hpclub/RELeARN>

Step 2 Create and enter a build directory inside `relearn-master/relearn`.

Step 3 Configure the build with the Score-P compiler wrappers, with Score-P instrumentation disabled for the configure step itself, RELeARN's own (source-level) Score-P integration disabled since instrumentation is instead provided by the compiler wrappers, and CUDA support disabled:

```
SCOREP_WRAPPER=off CC=scorep-gcc \
  CXX=scorep-g++ cmake .. \
  -DENABLE_SCOREP=OFF -DENABLE_CUDA=off
```

Step 4 Build the `relearn` target, enabling OpenMP instrumentation via `Opari2` (`--thread=omp`), skipping source annotation (`--opari=--nosrc`), and disabling OpenCL instrumentation (`--noopencl`):

```
SCOREP_WRAPPER_INSTRUMENTER_FLAGS= \
  '--thread=omp --noopencl \
  --opari=--nosrc' make -j48 relearn
```

Artifact Execution

Data collection for A_8 is split into generating the measurement orchestration script and then running it. The workflow is T_1 (build & configure) $\rightarrow T_2$ (generate script) $\rightarrow T_3$ (run measurement sweep) $\rightarrow T_4$ (evaluate).

- T_1 Build RELeARN with the Score-P compiler wrappers as described above.
- T_2 Generate the measurement orchestration script (`norc_generate`) from the `relearn_job_template.sh` job template, using the counters from the MQM `selected_counters.json` produced for A_5 (`--counter-set both`). Sweep the neurons-per-rank parameter N over `--var neurons=250,500,750,1000,1250` and the number of MPI ranks p over `--var ntasks=2,4,8,16,32,64`, with a single repetition per counter combination (`--iterations 1`).
- T_3 Run the generated script to execute the full parameter sweep, including the held-out reference point, collecting the raw counter data in the results directory. Additionally, measure the held-out reference point `--var neurons=1500, --var ntasks=64`, used to evaluate the mean relative error against ground truth.
- T_4 Model the results with Extra-P; specifically, run the RELeARN evaluation script (`relearn_evaluation.py`) to obtain the mean exponent deviation of the models, and the `relative_error_evaluation.py` script to determine the relative errors. Both scripts run the Extra-P base and HWC modelers on the dataset.

Artifact Analysis (incl. Outputs)

The evaluation scripts produce CSV files containing the mean-exponent-deviation and relative-error data shown in Fig. 8.

I. Computational Artifact A_9

Relation To Contributions

Artifact A_9 is the measured data obtained on the Grace Hopper system for the Kripke case study, collected for the demonstration C_3 and modeled with C_2 .

Expected Results

Modeling the collected Kripke measurements with the Extra-P base and HWC modelers should reproduce the mean exponent deviation and relative-error results shown in Fig. 6, showing that Extra-P, combined with NORC-selected counters, produces noise-resilient performance models for real applications.

Expected Reproduction Time (in Minutes)

Data acquisition: around 7100 min. The evaluation scripts (relative error and mean exponent deviation) take roughly 30 min combined across all of A_7 – A_{10} .

Artifact Setup (incl. Inputs)

Hardware: A Grace Hopper Superchip node (72 Neoverse V2 Armv9 cores, 480 GB LPDDR5X, NVIDIA GH200 GPU with 96 GB HBM3).

Software: Kripke¹⁰ (benchmark code, bundling RAJA as `tpl/raja`), Score-P (for instrumentation and profiling), and Extra-P (A_2) for modeling.

Datasets / Inputs: A_9 is generated by running the measurement orchestration scripts described below with the parameter sets and job templates provided in the dataset.

Installation and Deployment: On the Grace Hopper system, Kripke is set up via Spack rather than environment modules; the provided `setup_kripke_spack.sh` script automates the steps below.

- Step 1 Install and load the required dependencies via Spack: `openmpi`, `cmake`, and `scorep`, using the system's default compiler.
- Step 2 Download the official Kripke v1.2.7 release bundle (`kripke-v1.2.7-ddcac43.tar.gz`), which includes RAJA.
- Step 3 Point the compiler variables to the Score-P instrumented wrappers: `CC=scorep-mpicc`, `CXX=scorep-mpic++`, `FC=scorep-mpif77`.
- Step 4 Apply the same RAJA `scan.hpp` patch described for A_7 .
- Step 5 Configure and build as on Lichtenberg II, with `SCOREP_WRAPPER=off` for the configure step and `SCOREP_WRAPPER_INSTRUMENTER_FLAGS="--thread=none"` for the build.

Artifact Execution

Data collection for A_9 is split into generating the measurement orchestration script and then running it. The workflow is T_1 (build & configure) $\rightarrow T_2$ (generate script) $\rightarrow T_3$ (run measurement sweep) $\rightarrow T_4$ (evaluate).

¹⁰<https://github.com/LLNL/Kripke>

- T_1 Build Kripke with the Score-P compiler wrappers as described above.
- T_2 Generate the measurement orchestration script (`norc_generate`) from the `run_kripke.sh` job template, using the counters from the MQM `selected_counters.json` produced for A_6 (`--counter-set both`). Sweep the group-size parameter G over `--var grp=24,32,40,48,56`, the zones-per-rank parameter Z_p over `--var edge=4,8,12,16,20`, and the number of MPI ranks p over `--var ntasks=2,3,4,5`, with a single repetition per counter combination (`--iterations 1`).
- T_3 Run the generated script to execute the full parameter sweep, including the held-out reference point, collecting the raw counter data in the results directory. Additionally, measure the held-out reference point `--var grp=64, --var edge=24, --var ntasks=32`, used to evaluate the mean relative error against ground truth.
- T_4 Model the results with Extra-P; specifically, run the Kripke evaluation script (`kripke_evaluation.py`) to obtain the mean exponent deviation of the models, and the `relative_error_evaluation.py` script to determine the relative errors. Both scripts run the Extra-P base and HWC modelers on the dataset.

Artifact Analysis (incl. Outputs)

The evaluation scripts produce CSV files containing the mean-exponent-deviation and relative-error data shown in Fig. 6.

J. Computational Artifact A_{10}

Relation To Contributions

Artifact A_{10} is the measured data obtained on the Grace Hopper system for the RELeARN case study, collected for the demonstration C_3 and modeled with C_2 .

Expected Results

Modeling the collected RELeARN measurements with the Extra-P base and HWC modelers should reproduce the mean exponent deviation and relative-error results shown in Fig. 8, showing that Extra-P, combined with NORC-selected counters, produces noise-resilient performance models for real applications.

Expected Reproduction Time (in Minutes)

Data acquisition: around 470 min. The evaluation scripts (relative error and mean exponent deviation) take roughly 30 min combined across all of A_7 – A_{10} .

Artifact Setup (incl. Inputs)

Hardware: A Grace Hopper Superchip node (72 Neo-verse V2 Armv9 cores, 480 GB LPDDR5X, NVIDIA GH200 GPU with 96 GB HBM3).

Software: RELeARN¹¹ (benchmark code), Score-P (for instrumentation and profiling), and Extra-P (A_2) for modeling.

Datasets / Inputs: A_{10} is generated by running the measurement orchestration scripts described below with the parameter sets and job templates provided in the dataset.

Installation and Deployment: On the Grace Hopper system, RELeARN is set up via Spack rather than environment modules; the provided `setup_relearn_spack.sh` script automates the steps below.

- Step 1 Install and load the required dependencies via Spack, pinned to `gcc@14: openmpi, cmake, scorep@9.4`, and `boost` (built with the `+json +random` variants).
- Step 2 Clone the `tuda-parallel/RELeARN` fork (main branch).
- Step 3 Point the compiler variables to the Score-P instrumented wrappers: `CC=scorep-mpicc, CXX=scorep-mpic++`.
- Step 4 Configure the build with `SCOREP_WRAPPER=off, -DENABLE_SCOREP=OFF`, and `-DENABLE_CUDA=off`.
- Step 5 Build the `relearn` target with `SCOREP_WRAPPER_INSTRUMENTER_FLAGS="--thread=omp --opari=--nosrc"`.

Artifact Execution

Data collection for A_{10} is split into generating the measurement orchestration script and then running it. The workflow is T_1 (build & configure) $\rightarrow T_2$ (generate script) $\rightarrow T_3$ (run measurement sweep) $\rightarrow T_4$ (evaluate).

- T_1 Build RELeARN with the Score-P compiler wrappers as described above.
- T_2 Generate the measurement orchestration script (`norc_generate`) from the `relearn_job_template.sh` job template, using the counters from the MQM `selected_counters.json` produced for A_6 (`--counter-set both`). Sweep the neurons-per-rank parameter N over `--var neurons=250,500,750,1000,1250` and the number of MPI ranks p over `--var ntasks=1,2,4,8,16`, with a single repetition per counter combination (`--iterations 1`).
- T_3 Run the generated script to execute the full parameter sweep, including the held-out reference point, collecting the raw counter data in the results directory. Additionally, measure the held-out reference point `--var neurons=1500, --var ntasks=32`, used to evaluate the mean relative error against ground truth.
- T_4 Model the results with Extra-P; specifically, run the RELeARN evaluation script (`relearn_evaluation.py`) to obtain the mean exponent deviation of the models, and the `relative_error_evaluation.py` script to

¹¹<https://github.com/tuda-hpclab/RELeARN>

determine the relative errors. Both scripts run the Extra-P base and HWC modelers on the dataset.

Artifact Analysis (incl. Outputs)

The evaluation scripts produce CSV files containing the mean-exponent-deviation and relative-error data shown in Fig. 8.